

Actividad I

Microprocesadores

Diseño de un sistema basado en el microprocesador Z80

Vicente Alfonso Torres Razo

19 de septiembre de 2026

Resumen

Este documento presenta el diseño de un sistema digital basado en el microprocesador Z80, capaz de almacenar 16 KB de código en una memoria EEPROM 28C256, 16 KB de datos en una memoria SRAM 6264, y comunicarse con dispositivos externos a través de 6 puertos paralelos implementados con circuitos integrados 8255 (PPI). Se incluye el mapa de memoria y de puertos, el diagrama electrónico elaborado en KiCad, y un programa en lenguaje ensamblador que solicita al usuario su nombre y apellidos, valida los caracteres introducidos y cuenta la cantidad de letras ingresadas.

Introducción

Un sistema basado en microprocesador requiere, además de la unidad central de procesamiento (CPU), memoria para almacenar instrucciones y datos, así como interfaces que permitan la comunicación con el exterior. El diseño de estos sistemas implica decidir cómo se organiza el espacio de direcciones disponible, de forma que cada dispositivo (memoria o puerto) responda únicamente dentro del rango que le corresponde, evitando conflictos eléctricos entre dispositivos. En este trabajo se diseña un sistema mínimo basado en el Z80 que cumple con los requerimientos de capacidad y direccionamiento especificados en la actividad.

1. Funcionamiento del sistema

1.1. Microprocesador

El Z80 es un microprocesador de 8 bits con un bus de direcciones de 16 líneas (A0-A15), capaz de direccionar hasta 64 KB de memoria, y un bus de datos bidireccional de 8 líneas (D0-D7). Distingue entre accesos a memoria y accesos a puertos de entrada/salida mediante las señales de control **MREQ** (Memory Request) e **IORQ** (I/O Request), y entre lectura y escritura mediante **RD** y **WR**. Estas señales, junto con las líneas de dirección, son la base de la lógica de decodificación utilizada en este diseño.

1.2. Memorias

El sistema utiliza dos tipos de memoria: una EEPROM 28C256, de 32 KB de capacidad, de la cual se emplean únicamente 16 KB para almacenar el código del programa; y una SRAM 6264, de 8 KB de capacidad, empleada para el almacenamiento temporal de datos durante la ejecución.

La selección de cada chip se realiza mediante lógica combinacional que decodifica las líneas más significativas del bus de direcciones.

1.3. Interfaz para periféricos

Para satisfacer el requerimiento de 6 puertos de comunicación paralela, se utilizan dos circuitos integrados 8255 (PPI, Programmable Peripheral Interface). Cada 8255 proporciona 3 puertos de 8 bits (A, B y C) configurables como entrada o salida mediante una palabra de control, por lo que dos de estos circuitos satisfacen exactamente el requerimiento de 6 puertos.

2. Diseño del sistema

2.1. Mapa de memoria y puertos

Rango	Dispositivo	Función
0000h – 3FFFh	28C256	Memoria de código
F800h – FFFFh	6264	Memoria de datos

Cuadro 1: Mapa de memoria

Dirección	Registro	Chip
40h	Puerto A	8255 #1
41h	Puerto B	8255 #1
42h	Puerto C	8255 #1
43h	Control	8255 #1
44h	Puerto A	8255 #2
45h	Puerto B	8255 #2
46h	Puerto C	8255 #2
47h	Control	8255 #2

Cuadro 2: Mapa de puertos

2.2. Diagrama electrónico

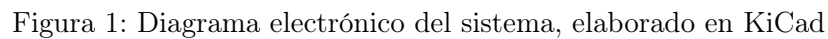
2.3. Programa en ensamblador

A continuación se presenta el código fuente del programa, escrito en lenguaje ensamblador para Z80 y probado en la herramienta Z80_workbench.

```

1 ; Mapa de memoria:
2 ;   0000h - 3FFFh   EEPROM 28C256 (codigo)
3 ;   F800h - FFFFh   SRAM 6264 (datos)
4 ; Mapa de puertos:
5 ;   40h  Puerto A del 8255 #1   (entrada, teclado)
6 ;   41h  Puerto B del 8255 #1   (salida, pantalla)
7 ;   42h  Puerto C del 8255 #1
8 ;   43h  Registro de control del 8255 #1
9 ;   44h a 47h  8255 #2 (reservado para expansion)
10 ; Uso de la memoria de datos:

```



3

```

30
31 CAPTURA :
32     IN A, (40h)                ; lee un caracter del teclado
33
34     CP 0Dh                     ; Enter termina la captura
35     JP Z, RESULTADO
36
37     CP 20h                     ; el espacio es valido pero no es letra
38     JP Z, GUARDA
39
40     CP 41h                     ; menor que 'A'
41     JP C, ERROR
42     CP 5Bh                     ; de 'A' a 'Z'
43     JP C, LETRA
44     CP 61h                     ; entre 'Z' y 'a' hay simbolos
45     JP C, ERROR
46     CP 7Bh                     ; de 'a' a 'z'
47     JP C, LETRA
48     JP ERROR                   ; cualquier otro caracter
49
50 LETRA :
51     PUSH AF                    ; guarda el caracter
52     LD A, (0F829h)
53     INC A
54     LD (0F829h), A              ; una letra mas
55     POP AF                     ; recupera el caracter
56
57 GUARDA :
58     LD (HL), A                 ; guarda el caracter en el buffer
59     INC HL
60     OUT (41h), A               ; lo muestra en pantalla
61
62     LD A, (0F828h)
63     INC A
64     LD (0F828h), A              ; un caracter mas guardado
65     CP 40                       ; si el buffer se lleno, termina
66     JP Z, RESULTADO
67     JP CAPTURA
68
69 ERROR :
70     LD DE, MENSAJE2
71     CALL IMPRIME
72     JP CAPTURA                 ; vuelve a pedir un caracter
73
74 RESULTADO :
75     LD DE, MENSAJE3
76     CALL IMPRIME
77
78     LD A, (0F828h)
79     LD B, A                     ; B = cuantos caracteres hay
80     LD DE, 0F800h               ; DE recorre el buffer
81
82 MUESTRA :
83     LD A, B

```

```

84      OR A
85      JP Z, CUENTA          ; ya se imprimieron todos
86      LD A, (DE)
87      OUT (41h), A
88      INC DE
89      DEC B
90      JP MUESTRA
91
92 CUENTA:
93      LD DE, MENSAJE4
94      CALL IMPRIME
95
96      LD A, (0F829h)        ; total de letras
97      LD B, 0               ; B contara las decenas
98
99 DECENAS:
100     CP 10
101     JP C, DIGITOS         ; ya quedan menos de 10
102     SUB 10
103     INC B
104     JP DECENAS
105
106 DIGITOS:
107     LD C, A               ; C = unidades
108     LD A, B
109     OR A
110     JP Z, UNIDADES        ; si no hay decenas, no las imprime
111     ADD A, 30h            ; convierte a ASCII
112     OUT (41h), A
113
114 UNIDADES:
115     LD A, C
116     ADD A, 30h
117     OUT (41h), A
118
119 FIN:
120     HALT
121     JP FIN
122
123 IMPRIME:
124     LD A, (DE)
125     OR A
126     RET Z
127     OUT (41h), A
128     INC DE
129     JP IMPRIME
130
131
132 MENSAJE1:
133     .TEXT "Escribe tu nombre y apellidos: "
134     .DB 0
135
136 MENSAJE2:
137     .DB 0Dh, 0Ah

```

```
138     .TEXT "ERROR: solo se permiten letras y espacios"
139     .DB 0Dh, 0Ah, 0
140
141 MENSAJE3:
142     .DB 0Dh, 0Ah
143     .TEXT "TEXTO CAPTURADO:"
144     .DB 0
145
146 MENSAJE4:
147     .DB 0Dh, 0Ah
148     .TEXT "NUMERO DE LETRAS:"
149     .DB 0
150
151     .END
```

3. Análisis de los resultados

El programa fue ensamblado exitosamente utilizando la herramienta Z80_workbench, obteniendo el archivo en formato HEX correspondiente. Su comportamiento fue verificado ejecutándolo en modo paso a paso (*STEP*), observando el cambio en los registros del procesador ante distintos valores de entrada.

Conclusión

Haciendo esta actividad me di cuenta que diseñar un sistema con microprocesador no es solo saber programar, sino que hay que pensar también en cómo se conecta todo físicamente, o sea la parte de hardware. Sinceramente se me hizo complicado entender por qué se necesitaba una lógica de decodificación de direcciones, pero conforme fui avanzando entendí que sin eso los chips de memoria y los puertos entrarían en conflicto porque todos ^{es}cucharían. ^al mismo tiempo.

Lo más difícil para mí fue la parte de simular el programa, ya que la herramienta Z80_workbench tiene su propio hardware fijo (una terminal con puertos ya definidos) que no necesariamente coincide con el mapa de direcciones que yo diseñé para este sistema. Por eso decidí comprobar el funcionamiento del programa usando el modo paso a paso (*STEP*), verificando manualmente que la lógica de validación de caracteres y el conteo de letras se comportara como se esperaba.

En general creo que el ejercicio me sirvió para relacionar lo que se ve en la materia de manera teórica (mapas de memoria, señales de control, direccionamiento) con una implementación más concreta, aunque todavía me falta práctica con KiCad para poder armar diagramas más complejos con mayor soltura.

Referencias

- Zilog, Inc. *Z80 CPU User Manual*.
- Microchip Technology. *28C256 EEPROM Datasheet*.
- Intel Corporation. *8255A Programmable Peripheral Interface Datasheet*.